VLA Training-Data Attribution

Proof-of-concept outline, evaluation methodology, method landscape, and a close reading of RoboDrop

Summary of discussion · September 2026

Goal
Build a proof of concept that explains **which training data** contributed to a good or bad Vision-Language-Action (VLA) training result. Scope is deliberately limited to data: model, config, training schedule and evaluation protocol are held fixed so that data is the only variable.

Differentiators

- **Clean data can still be harmful.** Per-sample compatibility cannot see harm that comes from the mixture (Pebblous: 93 correct episodes, 73% → 43%).
- **No trusted reference set needed.** Rollout outcomes are the reference, so it works on new collection campaigns and heterogeneous multi-source data where “clean” is undefined.
- **Diagnosis, not just filtering.** It names the episodes behind a specific failure on a specific task.
- **Objective dependence.** One score per metric; curating for throughput and for reliability selects different subsets of the same data.
- **Timing and permanence.** A warm-up-epoch verdict is permanent, yet data that conflicts early may matter in later epochs. Outcome scores are reversible and can reweight instead of delete.
- **It finds helpful data too**, which points at what to collect next. A filter only removes.
- **Headline novelty:** CUPID’s estimator, new application — per-metric and per-failure-type returns producing an episode × metric matrix.
- **Sharpest testable claim:** divergence on a single metric, exposing a filter’s implicit bias toward reliability over throughput.

Contents

1	The core question and scope
2	PoC design
3	Evaluation layer: PhAIL primitives
4	Metric-resolved attribution
5	Landscape of attribution and curation approaches
6	Reference papers
7	Industry landscape
8	RoboDrop in depth
9	Proposed combined architecture
10	First milestone and next steps
	References

Key takeaways

- Lineage (exact data versions, episode IDs, configs and checkpoints tied to eval results) is the foundation. Attribution is impossible without it.
- Attribute against **closed-loop rollout outcomes**, not training or validation loss. In imitation learning, low offline loss correlates poorly with task success.
- Adopt **PhAIL's evaluation primitives**: time-to-success distributions, hard failures as explicit events, a human-relative score, and a distributional significance test, so small effects are measurable at affordable rollout counts.
- Attribute **per metric**, not just overall: principled metrics can crown different winners on the same data. This is the PoC's main contribution — existing methods attribute a single notion of success.
- Validate the method with **planted ground truth**: corrupt a known subset of episodes and measure whether the method finds it; then confirm with counterfactual retraining.

1. The core question and scope

“Which artifacts contributed to a good or bad training result” can refer to data, configs, code, or checkpoints, and each requires different machinery. The PoC targets **training data only**, because most VLA quality problems trace back to the data mix and because data attribution is the most demonstrable result.

Attribution can be expressed at several levels of granularity:

Unit	Example	When it is useful
Data source	A dataset or collection campaign	Mixture decisions; cheapest to ablate
Task	“open drawer” demonstrations	Explaining per-task success differences
Episode	One demonstration	The natural atomic unit for curation and review
Segment / step	A few seconds inside an episode	Locally bad supervision inside otherwise good episodes

Per-episode scores become most valuable when aggregated by metadata. Findings such as “demos from operator 3 hurt drawer tasks” or “episodes over 400 steps are net harmful” are usually what stakeholders act on.

2. PoC design

2.1 Implementation roadmap

1. Build lineage before attribution. You can't attribute anything you haven't recorded. Every training run should be tied to exact dataset versions (content hashes), episode IDs, mixture weights, augmentation settings, the config, the git SHA, and the resulting checkpoints. Every eval rollout should be tied back to that checkpoint. W&B Artifacts, MLflow, or DVC handle this well. This part is unglamorous, but it's the backbone. It is also useful on its own, because it lets you answer "what changed between the good run and the bad run?"

2. Pick a small, fast, reproducible setup. Use simulation so evals are cheap and repeatable. LIBERO is a good default because it has distinct task suites, so failures are easy to localize. SimplerEnv is worth considering if you care about real-robot correlation. For the model, use something you can retrain many times: SmolVLA via LeRobot is small enough for a single GPU. Fine-tuning OpenVLA-OFT or π0 through openpi also works if you have more compute. Attribution methods often need many training runs, so iteration speed matters more than state-of-the-art performance.

3. Make evaluation statistically honest. Success rates in manipulation are noisy. Use enough rollouts per task (roughly 50 or more), fixed seeds and initial states, and report confidence intervals. Otherwise you'll "attribute" noise. Also log per-rollout outcomes with a simple failure taxonomy (missed grasp, wrong object, ignored instruction, timeout). "Which data caused wrong-object failures" is a far more useful question than "which data lowered the overall score."

4. Plant ground truth, which is the key trick for a PoC. Deliberately corrupt known subsets of training data: noisy or shifted actions, mismatched language instructions, altered camera viewpoints, or low-quality teleoperation demos. Then check whether your attribution method flags exactly those. This turns "the method says X" into "the method recovered 90% of the injected bad episodes." That's a convincing demo.

5. Layer attribution methods from cheap and causal to fine-grained.

- **Leave-one-group-out ablations.** Drop one data source or task group, retrain, and measure the change. This is coarse but unambiguous, and it gives you a reference result for everything else.

- **Randomized subset training with regression (datamodels-style).** Train many runs on random data subsets, then fit the effect of each group on success. This scales better than one-at-a-time ablations.
- **Gradient-based influence (TracIn, TRAK).** These give per-episode scores without retraining. There is recent robotics-specific work applying influence functions and mutual-information estimators to robot data curation (look up CUPID and DemInf from 2025). They're worth reading before you implement your own.
- **Embedding nearest-neighbor retrieval.** Link failed rollouts to similar training episodes. This is cheap and good for a UI, but it's correlational, so validate it against the ablations.

> **Connection to the evaluation layer**
> Step 3 is where PhAIL fits. Section 3 upgrades “success rate with confidence intervals” to PhAIL’s distributional methodology, which is better suited to detecting the small effects that attribution and counterfactual verification produce.

2.2 Episode catalog

Each episode gets a stable ID and a metadata row: source, task, language instruction, scene, objects, operator, collection date, episode length, and basic action statistics (smoothness, idle time, gripper toggles). LeRobot’s dataset format plus a Parquet metadata table works well. Per-episode attribution scores become most valuable when aggregated by these columns.

2.3 Experimental setup

Component	Recommendation	Why
Simulation benchmark	LIBERO (default); SimplerEnv if real-robot correlation matters	Cheap, repeatable; LIBERO suites make failures easy to localize
Real-robot benchmark	PhAIL (Franka FR3, bin-to-bin picking, four objects)	Distributional evaluation primitives, same-fixture human reference, fully published rollouts
Model	SmolVLA via LeRobot; OpenVLA-OFT or π0 (openpi) with more compute	Attribution needs many training runs; SmolVLA is also one of PhAIL’s baselined models
Controls	Fixed config and steps; 2–3 seeds per data configuration	Prevents seed variance from masquerading as a data effect

2.4 What counts as a good or bad result

The primary signal comes from closed-loop evaluation rollouts, scored with the PhAIL primitives in Section 3 and broken down by task or object and by failure mode. Loss on held-out demonstrations is acceptable for fast iteration, but final claims should be about closed-loop behavior.

> **Why offline metrics are not enough**
> Pebblous reported an experiment where adding 93 physically correct synthetic episodes to a GR00T training set, without reweighting, dropped closed-loop success from 73% to 43%, while offline prediction error stayed near-perfect in both conditions. The cause was an over-representation of near-idle demonstrations.

2.5 Corruption types for the injection test

Corruption type	Simulates
Noisy or time-shifted actions	Teleoperation lag, timestamp misalignment
Swapped or wrong language instructions	Annotation errors
Altered camera viewpoint or dropped frames	Sensor drift, hardware faults
Low-skill or hesitant demonstrations	Inexperienced operators, idle periods, corrections

Inject 5–10% corrupted episodes and measure how many the method ranks near the top.

2.6 Counterfactual verification

- Remove the top-k “harmful” episodes and retrain: the policy should improve more than after removing k random episodes.
- Remove the top-k “helpful” episodes and retrain: the policy should degrade more than after random removal.
- Each of these is a close comparison between two policies, so use the PhAIL significance gate (Section 3.4) to decide whether the change is real before claiming it.

2.7 Demonstration surface

A dashboard: select a failed rollout, see its failure type, the most influential training episodes (with video) and their source datasets; or browse a ranked list of harmful episodes, aggregated by any catalog column. PhAIL’s public run explorer, which shows each rollout with synchronized video, telemetry and event annotations, is a good design reference for the rollout side of this view.

3. Evaluation layer: PhAIL primitives

PhAIL (Physical AI Leaderboard; Arkhangelskiy, Positronic Robotics, arXiv:2605.29710, May 2026) is an open real-robot VLA benchmark and a distributional evaluation methodology. It is a natural fit for this PoC in two roles: its **primitives** become the evaluation layer that attribution is measured against, and its **station and data** provide a real-robot stage after simulation.

3.1 The benchmark

Aspect	Detail
Hardware	Franka Research 3 with Robotiq 2F-85 gripper; over-the-shoulder and wrist RGB cameras (DROID-style)
Task	Bin-to-bin order picking of four objects: wooden spoons (rigid, elongated), towels (deformable), scissors (articulated), batteries (small, rigid). Each rollout contains many placement operations
Models	OpenPI π0.5, GR00T N1.6, ACT and SmolVLA, all fine-tuned on the same 449-episode (~13 h) demonstration set with each repository's default recipe
Data	~990 evaluation episodes including 396 same-fixture human-teleoperation reference rollouts; every rollout published with synchronized video, telemetry and event annotations
Protocol	Blind evaluation: a scheduler randomly picks which model runs next, the operator does not know which one it is, and only intervenes with safety stops
Tooling	Open-source positronic framework runs both the human reference and every model on the same hardware; analysis pipeline published on GitHub

3.2 The evaluation primitives

Primitive	Definition	What it fixes
Time-to-success T	Wall-clock time to complete one operation; each rollout yields several	Carries speed and reliability together; multiplies sample size
Ghost events ($T = \infty$)	Unrecoverable outcomes — item lost outside workspace, dropped and never collected, safety stop — count as completed at infinite time	The gap between the CDF's asymptote and 1 is exactly the hard-failure rate
Right-censoring	Timeouts are slow-but-recoverable, recorded as censored, not failed	Stops "slow" from being conflated with "broken"
Kaplan–Meier CDF	$F(t) = P(T \le t)$ estimated with censoring	Success@τ, median, RMST, UPH, completion, MTBF/A are all projections of F
HRT	Human-Relative Throughput: $RMST_{human}$ / $RMST_{model}$ per object, macro-averaged	Dimensionless; same-fixture reference partly cancels object difficulty
Clustered bootstrap	95% CIs resampling whole episodes, not operations	Operations in one episode are correlated; per-operation resampling gives CIs that are too narrow
Macro-KS test	Two-sample Kolmogorov–Smirnov per object, averaged; p-values from pooled-resample bootstrap	A significance gate sensitive to any distributional difference
P-P plots	$F_{human}(t)$ vs $F_{model}(t)$; area = $P(T_{model} < T_{human})$	Makes distribution shape (tails, early successes) visible

A core principle is the separation of two jobs: **scoring** (a scalar with a confidence interval, used for ranking and direction) and **significance** (the macro-KS test, used only to decide whether two policies differ at all). KS cannot rank on its own; the paper shows its pairwise signs can be intransitive.

3.3 Headline evidence

- Macro-KS resolved GR00T vs. ACT at 25 and OpenPI vs. ACT at 30 rollouts per (model, object) cell, where binary success thresholds did not reach 80% detection within 30.
- A paired binary test needs roughly 600–1500 rollouts per cell to detect a 5-point difference, about 30× more at the paper's effect sizes. The paper notes the ratio is setup-specific; the method is what generalizes.
- Four principled scalars on the same data produced three different top-1 models. Choosing a headline scalar is a disclosed methodological commitment.
- The closest pair (OpenPI vs. GR00T) stayed unresolved at about 37 episodes per cell. The best model was roughly 7× slower per operation than the human; SmolVLA was clearly worst, with an 18.6% intervention rate.
- Equal hard-failure rates can hide different causes: ACT's were mostly drops, SmolVLA's mostly safety stops.

3.4 How PhAIL fits the attribution PoC

The following integration is our proposal; PhAIL itself is an evaluation benchmark and does not perform data attribution.

PoC need	PhAIL contribution
Verification gate. Counterfactual checks compare the full-data policy with a policy trained without the top-k episodes. These are close comparisons with small effects.	Macro-KS is the significance gate; the HRT or RMST confidence interval gives the direction. This makes the retrain-and-compare loop affordable, especially on real hardware.
Richer attribution targets. A single success scalar mixes different kinds of harm.	Attribute separately against regions of the CDF: ghost-event rate (which data causes drops and safety stops), RMST or HRT (which data causes slowness, e.g. hesitant or idle demos), and the censored tail.
Declared targets. Scalars can rank policies in opposite orders.	Every attribution result states which functional it explains, following PhAIL's disclosure principle.
Strata. Task-level breakdowns.	Per-object cells and macro-aggregation map directly to per-task attribution.
Finer rollout signal for CUPID-style influence on returns.	Operation-level (T, event) pairs give several outcome samples per rollout, e.g. a return based on negative time-to-success with a penalty for ghost events.
Trusted references for RoboDrop-style screening.	Same-fixture human-teleoperation rollouts are clean, on-distribution references.
Failure taxonomy.	Ghost-event types (drop outside workspace, uncollected drop, safety stop) and censored timeouts form an operational failure taxonomy.
Fair comparisons between ablated policies on hardware.	Blinded, randomized same-session scheduling.

3.5 Two ways to use PhAIL

A. As methodology in simulation (LIBERO). Our adaptation, not part of the paper:

- T = time (or steps) to task success per episode; for long-horizon LIBERO-10 tasks, per sub-task.
- Ghost events = unrecoverable states a simulator can detect directly (object off the table, out of reach); timeouts are censored.

- Human reference = durations of LIBERO’s original human-teleoperated demonstrations on the same tasks. This is approximate, since demos and rollouts don’t share initial states.
- Simulation makes rollouts cheap, so the KS efficiency gain matters less here, but the distributional targets and failure semantics carry over unchanged.

B. As the real-robot stage.

- The 449-episode fine-tuning set is small enough for retraining-based attribution, and four model families, including SmolVLA, are already baselined on it.
- Run the injection test on that set: corrupt known episodes, fine-tune, evaluate on the PhAIL station, and check whether attribution recovers them against real closed-loop outcomes.
- Use the human reference rollouts both as the HRT anchor and as the clean reference set for screening.

Practical caveats

Real-robot evaluation requires access to the PhAIL station (it is open to submissions and run as a consortium) or a replica built with the positronic framework. Confirm which parts of the dataset, including the fine-tuning set, are downloadable and under what license. Close comparisons still need roughly 25–45 episodes per (model, object) cell, so run broad attribution in simulation and verify only the top findings on hardware. PhAIL currently covers one task family; the paper notes that ~40% of episodes needed manual annotation review and that each model used its default fine-tuning recipe.

4. Metric-resolved attribution

One of PhAIL's findings changes what the PoC should measure. Among its contributions, the paper argues that several reasonable summaries of the same evaluation data can crown different winners, so the headline metric has to be declared openly. Each phrase of that claim is unpacked below.

"Same data, opposite rankings." They ran four models once. From that single batch of data they computed four different scores. Depending on which score you look at, a different model comes out on top. Nothing about the robots changed — only the arithmetic applied afterward.

"Principled aggregations of the same CDF." Every one of those scores is built from the same curves, just summarized differently. And none of them is a bad or unfair choice. Each is a reasonable thing to measure, used by real people in real papers. That's what makes the disagreement uncomfortable — you can't dismiss it by saying someone picked a silly metric.

"Disagree on top-1." They disagree specifically about who wins. Not about small details in the middle of the ranking — about first place, the number that goes in the headline.

"No single scalar suffices." So there's no one number that settles the question. Asking "which model is best?" doesn't have an answer until you say what "best" means — fastest on average, or most likely to win a given attempt, or most likely to finish within thirty seconds. Those are different questions with different winners.

"The choice of headline scalar is a disclosed methodological commitment." This is the prescription. Since picking a metric quietly decides the winner, picking one is a real decision, not a formatting detail — and you owe the reader that decision up front. Say which number you led with and why, rather than presenting your ranking as if it were just what the data said.

> **The short version**
> The winner depends on the yardstick, so tell people which yardstick you used.

4.1 The four yardsticks in PhAIL

Metric	Question it answers	Leader in PhAIL
Success within 30 s, F(30 s)	How often does an operation finish within thirty seconds?	GR00T
HRT (per-object macro mean)	How fast is the policy relative to a human, averaged over objects?	OpenPI π0.5
Global RMST	How long does an operation take on average, pooled over all objects?	GR00T (OpenPI a hair behind)
AUC vs. human	How likely is the policy to beat the human on a given attempt?	ACT

Four principled scalars, three different winners. The paper also shows a second trade-off: as the per-rollout time budget grows, throughput-oriented UPH falls while reliability-oriented MTBF/A rises.

4.2 The idea: attribute each metric separately

If the metric decides which *model* wins, it can also decide which *data* helps. Instead of one influence score per episode, the PoC computes an **influence vector**: one score per metric. The question changes from "is this episode good or bad?" to "which metric does this episode improve, and which does it degrade?" Results are reported as an episode × metric matrix.

Episodes then fall into four groups:

Group	Pattern across metrics	Curation action
Consistent helpers	Improve every metric	Always keep; candidates to collect more of
Consistent harmers	Degrade every metric	Always drop; likely true data errors
Trade-off episodes	Improve some metrics, degrade others	Keep or drop depending on the declared objective
Metric-specific	Move one metric, neutral on the rest	Explain specific behaviours, e.g. slowness or drops

Plausible trade-off examples, to be tested rather than assumed: slow, careful demonstrations might improve reliability while hurting throughput; fast, aggressive ones might do the reverse. The Pebblous result, where near-idle demos made a policy freeze, suggests hesitant data mainly moves time-based metrics.

4.3 Which methods can do this

Method family	Metric-resolved?	How
Retraining ablations / datamodels	Yes, at no extra rollout cost	Every retrained policy's rollouts yield all metrics at once; fit one regression per metric on the same runs
CUPID-style influence on returns	Yes, with one return definition per metric	Success within 30 s → binary return; RMST → negative capped time; AUC → win indicator against a sampled human time. HRT is a ratio, so it needs a first-order (linearized) approximation
Loss- or quality-based methods (RoboDrop, TRAK/QoQ on validation loss, DemInf)	No	They score against a single loss or quality measure, which has no notion of speed vs. reliability. Useful for screening, not for this question

This is a strong argument for outcome-grounded attribution: only methods that look at closed-loop rollouts can say which yardstick an episode moves.

> **What is borrowed and what is new**
> The **estimator** is CUPID's; the **way it is applied** is not. CUPID attributes against task success — one notion of good, one score per episode. Defining one return per metric, and per failure type, and reporting the resulting episode × metric matrix, is this PoC's own contribution. It is also the clearest differentiator from the existing work: neither CUPID nor RoboDrop says anything about *which* metric an episode helps or hurts, so neither can support objective-specific curation.

4.4 Statistics and disclosure

- **Two-step verdict per intervention.** First the macro-KS gate: did removing or adding this data change the time-to-success distribution at all? Then each metric's bootstrap CI gives the direction for that metric.
- **Crossing curves mean trade-offs.** When the gate fires but metrics disagree in sign, the CDFs cross — PhAIL's "differ, but neither is uniformly better" outcome. That is exactly the trade-off episode signature.
- **Multiple comparisons.** Four metrics multiply the number of claims; correct for it (e.g. Bonferroni or Holm across metrics) before labeling an episode as a trade-off.
- **Label every claim** with its metric and time cap τ, following PhAIL's disclosure principle.

- **Agreement check.** Report rank correlation (e.g. Kendall's τ) between the per-metric attribution vectors. High agreement means data quality is roughly one-dimensional for this task; low agreement means curation must be objective-specific.

4.5 Validation and the demo

- **Metric-targeted injections.** Extend the injection test with corruptions designed to hit specific metrics — for example, idle or hesitant segments aimed at time-based metrics, and noisy gripper release near placement aimed at drops. Check whether attribution assigns each corrupted group to the expected metric. These mappings are hypotheses the experiment should confirm.
- **"Same dataset, opposite curations."** Curate the dataset twice — once for HRT, once for reliability (e.g. F(30 s) or hard-failure rate) — retrain on each subset, and show that each curated policy wins on its own metric. This mirrors PhAIL's "same data, opposite rankings" finding on the data side and would be the headline result of the PoC.
- **Visualisation.** An episode × metric heatmap, and a scatter of throughput influence against reliability influence with the four groups as quadrants.

5. Landscape of attribution and curation approaches

Methods differ mainly in the signal they use and whether that signal is causally tied to closed-loop performance. Roughly ordered from most causal to cheapest:

Family	Signal	Rollouts?	Granularity	Cost	Examples
Retraining ablations	Change in success after removing a group	Yes	Source / task	High (1 run per group)	Leave-one-group-out
Datamodels	Regression of outcome on random data subsets	Optional (surrogate loss)	Episode / cluster	Several × a full training run	DataMIL
Influence on return	Influence functions over rollout outcomes	Yes	Episode / step	Moderate	CUPID, ATHENA
Influence on val. loss	Influence on loss of validation demos	No	Step → episode	Moderate	QoQ, TRAK, TracIn
Gradient compatibility	Gradient cosine vs. matched clean demos	No	Step → episode	One warm-up epoch	RoboDrop
Rollout classifier	Similarity to failed-rollout trajectories	Yes	Episode	Low–moderate	Demo-SCORE
Intrinsic quality	Mutual information, redundancy, smoothness	No	Episode	Low	DemInf, SCIZOR, heuristics
Mixture optimization	Robust optimization over source weights	No	Source	Moderate	Re-Mix
Retrieval	Embedding similarity to target trajectories	No	Sub-trajectory	Low	STRAP; nearest-neighbor

Key insight from the literature
In the CUPID paper, DemInf's scores matched human quality labels well, but policies trained on CUPID-curated data matched or outperformed DemInf-curated ones. Human perception of demonstration quality does not necessarily correspond to the data that maximizes policy success. This is the central argument for outcome-grounded attribution over heuristic quality scoring.

6. Reference papers

CUPID: Curating Data your Robot Loves with Influence Functions

Agia et al. · Stanford · CoRL 2025 · arXiv:2506.19121

Estimates the influence of each training demonstration on the policy's expected return, using evaluation rollouts and a REINFORCE-style estimator. Used to filter harmful demos and select new ones. With under a third of curated data, it reached state-of-the-art diffusion-policy results on RoboMimic, with similar hardware gains. Public MIT-licensed code built on the diffusion policy codebase.

Relevance: Closest match to the PoC; best starting point to fork.

DataMIL: Selecting Data for Robot Imitation Learning with Datamodels

Dass et al. · UT Austin / MIT / Stanford · ICLR 2026 · arXiv:2505.09603

Applies the datamodels paradigm to robotics. Replaces rollouts with a surrogate validation loss on target-task demos, uses temporal clustering to reduce noise, and co-trains with target data. Validated on 60+ tasks including selection from Open X-Embodiment. Estimation costs several times a single full training run.

Relevance: Reference for the retraining-based, causal end of the spectrum.

Quality over Quantity (QoQ)

2026 · arXiv:2603.09056

Defines data quality as each sample's contribution to reducing loss on validation demonstrations, estimated with influence functions; takes max influence across validation samples and averages scores within each trajectory.

Relevance: Simpler influence variant; useful baseline.

ATHENA: Accelerated Multi-Task Heterogeneous Influence Functions

June 2026 · arXiv:2606.16208

Scales influence functions to billion-parameter multi-task VLA fine-tuning using Kronecker-structured gradient projection and a low-rank Hessian approximation (about 313× faster). On RoboTwin 2.0 it matched or beat full-data fine-tuning with 50% of demos in simulation and 66.7% on six real-robot tasks. Focused on fine-tuning; pretraining data is future work.

Relevance: Reference for scaling attribution to real VLAs.

RoboDrop: Curating VLA Post-Training Data via Local Gradient Compatibility

Xu et al. · September 2026 · arXiv:2609.10021

Scores each post-training sample by gradient compatibility with matched clean validation samples during a one-epoch warm-up, aggregates per episode, and filters automatically. Real-robot success rose from 35.0% to 67.5%. See Section 8.

Relevance: Screening component; its corruption protocol is a template for our injection test.

DemInf: Robot Data Curation with Mutual Information Estimators

Hejna et al. · Stanford / Google DeepMind · RSS 2025 · arXiv:2502.08623

Unsupervised, offline scoring by each trajectory's contribution to state–action mutual information, using kNN estimates over VAE embeddings. Agrees with expert labels; 5–10% gains on RoboMimic.

Relevance: Cheap baseline the PoC must beat.

Demo-SCORE: Curating Demonstrations using Online Experience

Chen et al. · 2025 · arXiv:2503.03707

Trains a policy on all demos, then trains a classifier on its successful vs. failed rollouts and filters demonstrations that resemble failure trajectories.

Relevance: Rollout-grounded but correlational baseline.

PhAIL: A Real-Robot VLA Benchmark and Distributional Methodology

Arkhangelskiy · Positronic Robotics · May 2026 · arXiv:2605.29710

Open real-robot benchmark on a Franka FR3 using the time-to-success CDF as the evaluation primitive, with hard failures as infinite-time events, Human-Relative Throughput for scoring, and a macro-averaged KS test for significance. Resolved close model pairs at 25–30 rollouts per cell where binary metrics could not.

Relevance: Evaluation layer and real-robot stage of the PoC. See Section 3.

Re-Mix · SCIZOR · STRAP

arXiv:2408.14037 · arXiv:2505.22626 · arXiv:2412.15182

Re-Mix learns data-source mixture weights with distributionally robust optimization. SCIZOR removes low-quality and redundant data with self-supervised signals. STRAP retrieves sub-trajectories similar to a target task.

Relevance: Context for mixture-level and retrieval-based curation.

7. Industry landscape

No startup was found whose core product is performance-grounded data attribution for VLAs. Searches can miss stealth companies, so this finding is tentative. Adjacent activity:

Player	What they do	Relation to the PoC
Pebblous	Data-quality company; published a GR00T mixing experiment and a survey arguing that no one has closed the loop from rollout failures back to data	Validates the problem; closest in framing
NVIDIA NeMo Curator	Large-scale video deduplication, clustering and selection before training	Preprocessing only; no feedback from rollout outcomes
Label Studio	Robotics episode-review interface for human scoring of rollouts and demonstrations	Human review, not attribution
Positronic Robotics (PhAIL)	Operates the PhAIL real-robot leaderboard as an open consortium (founding partners Nebius and Toloka); publishes every rollout with video and telemetry	Evaluation methodology and real-robot testbed; see Section 3

The absence of a dedicated product is an opening for the PoC's positioning.

8. RoboDrop in depth

> **Source note**
> Details below are based on the RoboDrop abstract, introduction and results excerpts, and third-party summaries. The full method section could not be retrieved, so the exact matching procedure, aggregation formula, thresholding rule, and checkpoint handling should be verified in the paper before implementing.

8.1 The question it answers

RoboDrop asks: *does this episode's supervision agree with trusted clean demonstrations?* "Supervision" means the observation→action pairs that tell the model what to do in each situation. It is a filter applied **before the main post-training run** (not a pre-training method); its reference point is a validation set you already trust, and its output is a keep/drop decision per episode.

8.2 Inputs and outputs

Inputs	Outputs
Pretrained VLA to be post-trained. Architecture-agnostic: mean AUROC above 85% for π0.5-Base, GR00T and X-VLA; 80.1% even with a randomly initialized action expert.	**Per-sample scores**: gradient cosine similarity against matched references. Cosine outperforms the raw dot product.
Candidate post-training dataset for the new task or embodiment, possibly containing execution mistakes, sensor drift or timestamp misalignment.	**Episode-level scores** aggregated from sample scores.
Clean validation (reference) set of trusted demos for the target task.	**Keep/drop decisions** from an automatic rule that sets the filtering budget itself, without knowing the contamination rate.
Compute for one warm-up epoch.	**Curated dataset** used for the real post-training run.

8.3 Reference samples

A reference sample is one observation→action pair from the clean validation demos, chosen to match the training sample being scored. Matching is both **task-semantic** (same task or instruction) and **visual** (similar scene or robot state). This is what "local" and "context-conditioned" mean: two correct samples from different situations can have very different gradients, so only a context-matched comparison makes disagreement meaningful.

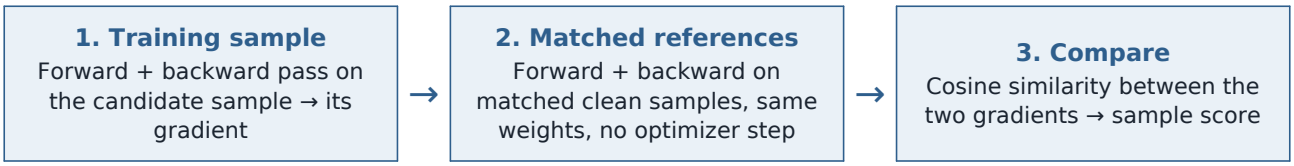
References are **not** derived from pre-training data. They are a small, held-out set of clean demonstrations for the target task, separate from the candidate data being audited.

8.4 Agreement is good

Gradient relationship	Interpretation	Decision
High cosine similarity	The sample pushes weights in the same direction as clean data; it teaches the right thing	Keep
Low or negative similarity	The sample pulls the model away from clean behavior — the signature of bad supervision	Drop

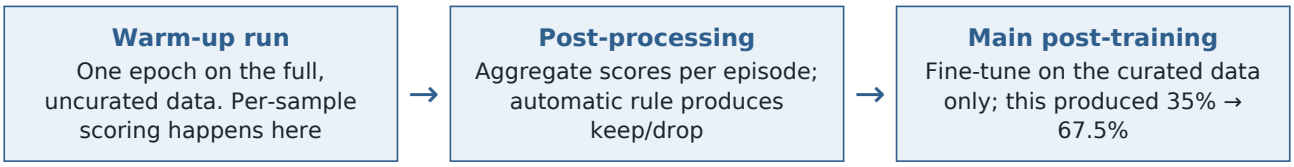
8.5 How reference gradients are obtained

Reference gradients are computed **on the fly during the warm-up run**, using the same model weights as the training sample being scored. A gradient is only meaningful relative to the current weights, which is why the paper describes the score as measured along the training trajectory.



Performed repeatedly during the warm-up epoch

8.6 Where the pipeline sits relative to post-training



Open question: whether the main run restarts from the original pretrained checkpoint or continues from the warm-up weights. Restarting is the cleaner design, since the warm-up model has already trained on the bad data.

8.7 Who curates the references

The team applying the method. RoboDrop assumes the trusted set exists; one reviewer flagged this assumption as untested. The paper's setups (clean vs. corrupted in simulation, expert vs. non-expert on the real robot) suggest the authors used known-good data from expert operators or uncorrupted sources.

Option	How	Trade-off
Dedicated expert sessions	Best teleoperator records careful demos per task	Most reliable; costs operator time
Manual review	Watch existing episodes and mark clearly good ones	Uses existing data; slow
Pre-filter + spot check	Shortlist via DemInf, success, smoothness; human confirms	Scales; heuristics can bias the set
Benchmark demos (PoC)	Treat LIBERO's original demos as clean; corrupt only the training copy	Free references plus ground truth
Same-fixture human teleop (real robot)	Use PhAIL-style human reference rollouts recorded on the evaluation fixture	Clean and on-distribution; human pace and strategy may differ from demos

Risks to manage

The reference set defines "correct." If it reflects one operator's style, valid alternative strategies may be scored as disagreement. If it lacks coverage of some visual states, samples from those states have no good match. Aim for several operators and broad state coverage per task.

8.8 Practical implementation concerns

These are standard for gradient-based methods and not confirmed details of RoboDrop:

- **Cost:** full per-sample gradients of a billion-parameter VLA are large. Restrict to a parameter subset (action expert, last layers, or LoRA weights) and compress with random projections; ATHENA is a good reference.
- **Stochastic loss:** flow-matching action heads sample noise and timesteps. Use the same noise and timestep for a training sample and its references.
- **Caching:** compute reference gradients at intervals and reuse them for nearby steps, accepting some staleness.
- **Code:** no public release was found; implementation may need to follow the paper.

8.9 RoboDrop versus the PoC

Dimension	RoboDrop	This PoC
Question	Does data conflict with trusted data?	Did this data cause this result?
Reference	Clean validation demos	Evaluation rollouts and success rates
Link to failures	None — never looks at rollouts, so it filters data but cannot diagnose behaviour	Core deliverable: “the policy grabs the wrong object on task 3, and these 40 episodes are why”, per task and failure mode
Reference set required	Yes — clean validation demos for the target task. The assumption fails for a new collection campaign or a heterogeneous multi-source dataset where “clean” is not defined	No — rollout outcomes are the reference, so it works where no trusted subset exists
Reference coverage	The reference set must span the training distribution, which is hard to achieve. Where it does not, matching still returns the *nearest* reference — a poor match — and the gradients diverge because the contexts differ, not because the sample is bad. It is dropped anyway, and this penalizes exactly the rare or novel regions the policy has the most to learn from	Rollouts cover whatever the policy actually does, so there is no coverage requirement on a curated subset; an episode is judged by its effect, not by how close it sits to a reference
Timing of the decision	Scores come from one early warm-up epoch and the drop is permanent. A sample that conflicts with the model’s early state may be exactly what it needs in later epochs — recovery behaviour, or a hard case it has not learned yet — and once dropped it can never contribute	Scores come from the finished policy’s behaviour, so they reflect what the data did over the whole run. Decisions are reversible and can be re-estimated per checkpoint, or used to reweight rather than delete
Mixture effects	Per-sample scoring may miss harm that only arises from the mix	Group ablations and outcome attribution capture it
Objective dependence	One keep/drop decision, the same whatever you are optimizing for. It cannot say that an episode makes the policy more reliable but slower	One score per metric. Optimizing for throughput on a production line and for minimizing dropped items select different subsets of the same data (Section 4)
Helpful data	Primarily a filter for harmful data	Also identifies what successes depend on
Strengths	Cheap (one epoch), no rollouts, catches local errors inside good episodes	Causal, explains outcomes

8.10 The RoboDrop comparison experiment

RoboDrop's premise is that gradient conflict with clean references *is* the signal for bad supervision, and its experiments support that in settings where the two coincide by construction: controlled corruptions, non-proficient demonstrations and real collection errors. Corrupted data is bad *and* disagrees with clean references. What those settings cannot show is what happens when "conflicts early" and "harms the policy" come apart — rare recovery behaviour, valid but unusual strategies, or hard cases the warm-up model has not learned yet.

The experiment that settles it: run RoboDrop, keep the episodes it drops, then use outcome attribution and counterfactual retraining to check whether any of them were actually helpful, and whether anything it kept was actually harmful. Both outcomes are useful — close agreement means gradient conflict is a good proxy for harm and cheap screening suffices, which is a valuable negative result; divergence yields a concrete list of misclassified episodes and makes the case for outcome-grounded attribution directly.

> **Relation to CUPID, and what is actually new**
> This is structurally the same experiment CUPID ran, against a different opponent. CUPID compared itself with DemInf, an offline mutual-information filter; this compares against RoboDrop, a gradient-compatibility filter that uses the model, so it is the same shape of experiment applied to a newer and stronger baseline. Two differences: CUPID's comparison was largely about which curation produces the better policy, whereas this is a disagreement analysis that names the specific misclassified episodes; and the proxy under test is gradient compatibility rather than human-perceived quality. Because the general principle — outcome-grounded curation beats proxy-based curation — is already established by CUPID, this experiment is best framed as **supporting evidence**, with metric-resolved attribution (Section 4) as the headline claim, since neither CUPID nor RoboDrop says anything about which metric an episode helps.

The sharpest version: divergence on a single metric

A stronger target than general disagreement is divergence on one metric — for example: *RoboDrop dropped these 40 episodes; keeping them makes the policy 15% faster at no cost in reliability.* This is concrete, falsifiable, and beyond what any existing method can produce.

It is also more likely to hold, because gradient compatibility has a structural blind spot on speed. Compatibility measures whether a sample teaches the right action in a given state. A hesitant demo that eventually does the right thing can look compatible — its actions match what clean demos would do, merely stretched over more steps — while a fast, aggressive demo may push gradients in a direction a careful reference set does not, and be dropped. If so, RoboDrop is implicitly biased toward reliability and against throughput, and never says so, because it only emits keep/drop.

Experiment:

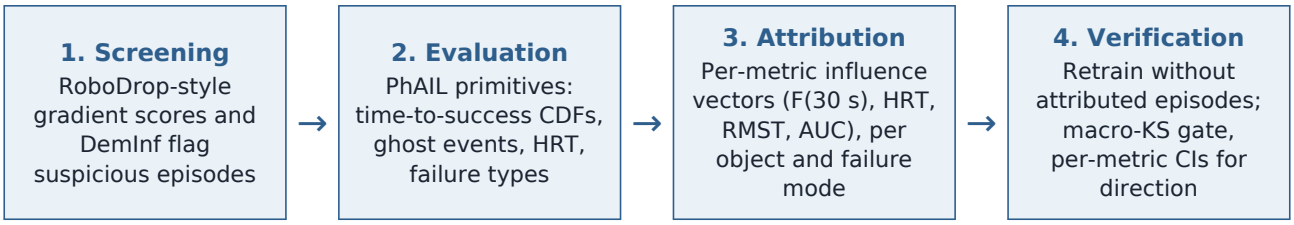
- Corrupt episodes two ways: pad some with idle or hesitant segments (should hurt speed, not reliability) and add gripper-release noise to others (should hurt reliability, not speed).
- Run RoboDrop and record what it keeps and drops.
- Attribute per metric on the same data.
- Test whether RoboDrop's decisions correlate more with the reliability attribution than with the speed attribution.

- **Verify before claiming.** Retrain with the disputed episodes restored and show the speed metric improves significantly (macro-KS gate plus the CI on RMST or HRT) while reliability does not degrade. Without this, the finding is an attribution artifact.

Three findings would follow: episodes RoboDrop drops that help throughput, episodes it keeps that slow the policy down, and the correlation asymmetry itself — evidence that a single-verdict filter silently commits to one objective.

9. Proposed combined architecture

RoboDrop, PhAIL and the attribution literature fit into the PoC as components:



9.1 The four steps in detail

Step 1 — Screening	
Input	The training dataset and a small set of trusted clean demos.
Procedure	1 Run DemInf over the dataset. Each episode gets a score for how consistent its actions are with what other demos do in similar states. 2 Run a short warm-up training pass (RoboDrop-style). Compare each sample's gradient with gradients from matched clean demos, and aggregate the results into an episode score. 3 Combine the two scores into a "suspicion" ranking.
Output	Every episode labeled with how bad it looks.
Why	It is cheap (no rollouts) and catches obvious errors early. Its verdicts are stored rather than acted on, so they can later be compared with what actually happened.

Step 2 — Evaluation	
Input	A policy trained on the dataset.
Procedure	1 Run many rollouts per task (e.g. 50) with fixed initial states. 2 For each rollout, record whether and when it succeeded, using the benchmark's goal check. 3 Classify failures with rules over simulator state: ghost events (unrecoverable, such as an object off the table), censored timeouts, and failure types (wrong object, missed grasp, idle). 4 Compute the metrics: time-to-success curves, success within 30 s, RMST, HRT against human demo durations, and AUC against humans.
Output	A detailed record of how the policy behaves, per task, per metric and per failure type.
Why	This is the ground truth. Attribution is only as good as this measurement.

Step 3 — Attribution	
Input	The training dataset, the trained policy, and the rollouts from Step 2.
Procedure	1 Define one “return” per metric for each rollout. For example, success within 30 s gives 1 or 0; for RMST, the return is negative time; for AUC, whether the rollout beat a sampled human time. 2 For each metric, CUPID-style influence estimates how much each training episode pushed that return up or down. 3 Repeat the analysis for subsets, such as a specific task or only rollouts with a specific failure type.
Output	An episode × metric table of influence scores, e.g. “episode 412 hurts success within 30 s and has no effect on speed.”
Why	This is the core of the PoC. It links behavior back to specific data, and separately for each metric.

Step 4 — Verification	
Input	The attribution rankings.
Procedure	1 Remove the top-k most harmful episodes and retrain. As a control, remove k random episodes and retrain. 2 Evaluate both policies as in Step 2. 3 Apply the macro-KS test: did the time-to-success distribution change at all? 4 If it did, check each metric’s confidence interval for the direction of the change. 5 Repeat for the top-k most helpful episodes (removing them should make things worse).
Output	Confirmation, or refutation, that the attribution is causally correct.
Why	Influence estimates can be wrong. This step turns them from claims into verified results.

9.2 What the architecture produces

The most compelling output is where screening and outcome attribution **disagree**: episodes that look bad but do not hurt the policy, and episodes that look clean but do. A second compelling output is **which kind** of harm each data group causes — drops and safety stops versus slowness — which a single success rate cannot show.

10. First milestone and next steps

Phase	Deliverable
Foundation	Lineage tracking; episode catalog; LIBERO (one or two suites) + SmolVLA
Evaluation layer	Kaplan–Meier time-to-success CDFs with ghost-event and censoring rules; HRT against LIBERO demo durations; episode-clustered bootstrap CIs; macro-KS test; P-P plots
Ground truth	Injection experiment with 5–10% corrupted episodes, using LIBERO's original demos as clean references
Causal baseline	Group-level leave-one-out ablations, judged with the macro-KS gate
Fine-grained attribution	Fork CUPID; attribute against ghost-event rate and RMST separately; add TRAK or RoboDrop-style scores; compare against DemInf
Metric-resolved attribution	Influence vector per episode across F(30 s), HRT, RMST and AUC; metric-targeted injections; "same dataset, opposite curations" experiment
Verification & demo	Counterfactual removal tests; dashboard linking failed rollouts to influential episodes
Real-robot stage	Injection test on the PhAIL fine-tuning set, evaluated on the PhAIL station or a positronic replica
Scale-up	Apply ATHENA/RoboDrop techniques to a larger VLA (e.g., π0.5 or GR00T)

Open decisions

- Compute budget — determines whether retraining-based attribution is feasible or gradient methods lead.
- Access to the PhAIL station or building a replica; availability and license of the fine-tuning set.
- Which metric is the declared headline objective for curation, and which are reported alongside it.
- Verify RoboDrop's method-section details (matching, aggregation, threshold, checkpoint handling).

References

1 Arkhangelskiy, S. PhAIL: A Real-Robot VLA Benchmark and Distributional Methodology. Positronic Robotics, 2026. arXiv:2605.29710. phail.ai; github.com/Positronic-Robotics/phail-paper
2 Agia, C. et al. CUPID: Curating Data your Robot Loves with Influence Functions. CoRL 2025. arXiv:2506.19121. Code: github.com/agiachris/cupid
3 Dass, S. et al. DataMIL: Selecting Data for Robot Imitation Learning with Datamodels. ICLR 2026. arXiv:2505.09603
4 Quality over Quantity: Demonstration Curation via Influence Functions for Data-Centric Robot Learning. 2026. arXiv:2603.09056
5 ATHENA: Accelerated Multi-Task Heterogeneous Influence Functions for Robot Data Curation. 2026. arXiv:2606.16208
6 Xu, R. et al. RoboDrop: Curating VLA Post-Training Data via Local Gradient Compatibility. 2026. arXiv:2609.10021
7 Hejna, J. et al. Robot Data Curation with Mutual Information Estimators (DemInf). RSS 2025. arXiv:2502.08623
8 Chen, A. S. et al. Curating Demonstrations using Online Experience (Demo-SCORE). 2025. arXiv:2503.03707
9 Hejna, J. et al. Re-Mix: Optimizing Data Mixtures for Large-Scale Imitation Learning. 2024. arXiv:2408.14037
10 Zhang, Y. et al. SCIZOR: Self-Supervised Data Curation for Large-Scale Imitation Learning. 2025. arXiv:2505.22626
11 Memmel, M. et al. STRAP: Robot Sub-Trajectory Retrieval for Augmented Policy Learning. ICLR 2025. arXiv:2412.15182
12 Liu, B. et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. NeurIPS 2023
13 Pebblous. Robot Data Curation & the Closed-Loop Gap. July 2026. blog.pebblous.ai